

A Better Universal ASIO Driver for DAWs

Configurable Buffer Size · Shared-Mode Multi-Client · Lower Latency Than FL Studio ASIO, FlexASIO, ASIO4ALL & Steinberg's Built-in Driver

Scope: a DAW-facing, user-mode ASIO driver only — not a system-wide audio driver, no kernel component, no OS-level replacement. Prepared August 2026.

The Core Finding

Your four complaints — FL Studio ASIO's latency, FlexASIO's latency, ASIO4ALL blocking other audio, and Steinberg's fixed buffer — all trace back to variations on the **same unclaimed technical gap**: none of the current universal ASIO drivers implement Windows' **IAudioClient3 low-latency shared-mode negotiation** API. This was directly confirmed while researching this document: the maintainer of PortAudio (the library FlexASIO is built on) has an open GitHub issue, filed December 2020 and **still open today**, explicitly asking for this exact feature — and FlexASIO's own author has stated on the record that PortAudio doesn't support it and he has no plans to add it himself. That is a real, verified, currently-unclaimed opening, not a guess.

Implementing that one API properly is the single highest-leverage thing this project can build: done right, it attacks all four named complaints at once, because it's the one mode that gets both low latency *and* lets other applications keep playing audio at the same time.

Validated Pain Points

Driver	Your complaint	What research confirms	Root cause
FL Studio ASIO	Too much latency	Image-Line's own bundled universal driver; user reports describe a "slight delay"; no published evidence it does per-device low-latency negotiation.	Conservative, one-size-fits-all generic backend, tuned for broad compatibility rather than tuned latency.
FlexASIO	Same latency issue as FL Studio	Ships defaulting to the legacy DirectSound backend out of the box. Even manually reconfigured to WASAPI, its underlying library (PortAudio) does not implement IAudioClient3's low-latency shared-mode calls — confirmed via an open PortAudio GitHub issue (#385, filed 2020, still open in 2026).	Shared mode is capped at the Windows audio engine's ~20 ms internal buffer; only switching to WASAPI Exclusive gets low latency (~3–4 ms), at the cost of losing multi-client audio.
ASIO4ALL	Can't stream other audio while it's active	Built on WDM-KS (Kernel Streaming). FlexASIO's own documentation states plainly that ASIO4ALL and other WDM-KS/exclusive-mode universal drivers always open the audio device in exclusive mode.	Architectural choice, not a bug — the same fundamental tradeoff as FlexASIO's WASAPI-Exclusive mode: lower latency in exchange for exclusive device ownership.
Steinberg built-in / generic driver	Buffer fixed (you observed 480 samples)	Steinberg's own help documentation confirms the buffer size and associated latency of the built-in driver "cannot be changed" by the user.	The driver's ASIOGetBufferSize call returns identical minimum/maximum/preferred values, so there is nothing for a host or control panel to select between.

PART 1 — RESEARCH

1.1 Why Each Competitor Has Its Specific Flaw — Research Results

The table above is the summary; here is the reasoning behind each row, since the fix for each problem is different and needs to be designed for specifically rather than assumed to share one cause.

FL Studio ASIO

Image-Line bundles its own universal ASIO driver so FL Studio works out of the box without a dedicated interface. Its internal backend choice isn't publicly documented in detail, but real-world reports are consistent with a conservative generic path rather than a per-device-tuned low-latency negotiation — the same category of behavior as Steinberg's bundled driver. It solves the “works on anything” problem, not the “lowest possible latency” problem.

FlexASIO

FlexASIO is the most technically capable of the four out of the box — it supports WASAPI Shared, WASAPI Exclusive, WDM-KS, DirectSound, and MME as selectable backends, and lets you set an exact buffer size via a configuration file. But its **default backend is DirectSound**, a legacy API, unless the user manually edits FlexASIO's TOML config file — something most users never discover. Even for users who do switch it to WASAPI, PortAudio's own maintainers have confirmed shared mode bottoms out around 20 ms because it never adopted IAudioClient3's low-latency negotiation calls; getting genuinely low latency out of FlexASIO today means switching to WASAPI Exclusive mode, which reintroduces the exact same single-app-only limitation as ASIO4ALL.

ASIO4ALL

ASIO4ALL is built on WDM-KS (Windows Kernel Streaming), a low-level API that talks close to the hardware but, like WASAPI Exclusive mode, takes exclusive ownership of the device. This is confirmed directly in FlexASIO's own documentation, which contrasts itself against ASIO4ALL/ASIO2KS on exactly this point. It is not a bug to be patched — it's the architecture ASIO4ALL chose, and every driver built the same way inherits the same limitation.

Steinberg built-in / Generic Low Latency ASIO Driver

Confirmed directly from Steinberg's own help documentation: the buffer size and associated latency of the built-in driver are fixed and cannot be changed by the user. Mechanically, this happens because the driver's ASIOGetBufferSize call — the protocol-level hook a host or control panel would use to offer a range — simply returns the same number for minimum, maximum, and preferred size, leaving nothing to select between.

1.2 ASIO Buffer-Size Protocol Mechanics — Research Results

- **ASIOGetBufferSize** — the driver reports minimum, maximum, preferred size, and granularity (the allowed step between sizes). This single call determines whether a driver offers a real range or a single fixed number.
- **ASIOCreateBuffers** — the host (or a control panel acting on its behalf) requests a specific size within that range; the driver allocates buffers accordingly.
- **Granularity** — 0 means any size is allowed; a negative value conventionally means powers-of-two only; a positive value is a literal step. A driver aiming for fine control should prefer 0 or a small step.

1.3 The Backend Tradeoff Table — Research Results

This is the central technical tradeoff every universal ASIO driver on the market today has picked one side of. Numbers below come directly from PortAudio/FlexASIO's own measured investigation into this exact question, plus Microsoft's published IAudioClient3 documentation.

Backend	Measured/documentated latency floor	Other apps can play audio too?	Used by
WASAPI Shared (standard)	~20 ms — the Windows audio engine's internal buffer size	Yes	FlexASIO (once switched off DirectSound), most consumer software
WASAPI Exclusive	As low as ~3–4 ms at small buffer sizes on capable hardware	No — device is locked to one app	FlexASIO (Exclusive option), ASIO2WASAPI
WDM-KS (Kernel Streaming)	~10 ms floor observed, otherwise follows requested buffer size closely	No — exclusive by design	ASIO4ALL, ASIO2KS
IAudioClient3 low-latency shared mode	Documented capability of ~1–3 ms in shared mode on WaveRT-capable hardware/drivers (per Microsoft's own low-latency audio documentation)	Yes — it is still shared mode	Not implemented by any surveyed universal ASIO driver as of this research (see 1.4)

IAudioClient3 shared mode is the only row in that table with both a low latency floor *and* multi-client support — and it is the one row nobody in the surveyed field has actually shipped.

1.4 The Unclaimed Gap — Research Results

Microsoft's IAudioClient3 interface (available since Windows 10) adds two calls specifically for this: **GetSharedModeEnginePeriod**, which reports the audio engine's true minimum/default period for a device, and **InitializeSharedAudioStream**, which opens a shared-mode stream at that tighter period instead of the conservative default. A 2020 PortAudio GitHub issue (#385) asked for exactly this to be added; as of this research it remains open, and a PortAudio maintainer noted the effort would need someone to actually implement it. Separately, FlexASIO's author confirmed on the record that PortAudio “doesn't seem to support that extended interface,” and that he had no plans to add it. JUCE framework contributors researching the same API independently described real-world shared-mode latencies of 2–3 ms as achievable on WaveRT-capable hardware once implemented correctly — calling it “the holy grail” of getting ASIO-class latency without needing a dedicated driver.

Caveat, stated honestly: **GetSharedModeEnginePeriod**'s actual achievable minimum is still hardware- and driver-dependent — it depends on the device's WaveRT miniport, exactly as established in the earlier phase of this project's research. Some devices (especially older or cheaper integrated codecs) will only report a modest improvement over the standard 20 ms shared-mode floor. This is a real, meaningful, unclaimed opportunity — not a guaranteed uniform result across all hardware.

1.5 Licensing Note

Since October 2025, Steinberg dual-licenses the ASIO SDK under GPLv3 in addition to the historical proprietary developer agreement, alongside moving the VST3 SDK to the MIT license. This is covered in full in Part 4 (Risks), since it materially affects how this driver can be licensed and distributed.

PART 2 — DESIGN

2.1 Scope and Architecture

Because this driver is DAW-facing only — not a system-wide replacement — the architecture is the simple end of what was evaluated in the earlier full research pass: a single user-mode COM component (a DLL registered under an ASIO CLSID), with no kernel driver, no WHQL signing, and no driver-store installation. This removes the hardest, slowest, and riskiest part of building a Windows audio driver entirely.

Component	What it does
ASIO shim (IASIO COM interface)	Implements init, getBufferSize, createBuffers, start/stop, getLatencies, etc. — the DAW-facing contract, per the ASIO SDK (now GPLv3/proprietary dual-licensed).
Backend negotiation layer	Tries IAudioClient3 low-latency shared mode first, falls back to WDM-KS, then standard WASAPI shared mode, per device (2.2).
Buffer-size protocol layer	Live-queries and reports the true min/max/preferred/granularity range per device and sample rate through ASIOGetBufferSize (2.3).
Real-time engine	Lock-free ring buffers, MMCSS “Pro Audio” thread registration, CPU-Sets P-core pinning — keeps a low, user-chosen buffer size stable instead of glitching (2.5).
Control panel	Standalone UI for backend/buffer-size selection, live range display, and safe-range warnings, for hosts that don't expose ASIO buffer selection cleanly.

2.2 Backend Negotiation Design — Attacking All Four Complaints at Once

Design: on stream open, the driver attempts backends in this order, per device, and remembers what worked:

- **1. IAudioClient3 low-latency shared mode** (default) — query GetSharedModeEnginePeriod for the device's true minimum period, then open with InitializeSharedAudioStream at that period. This is shared mode, so other applications keep working — directly solving the ASIO4ALL complaint — while targeting a latency floor meaningfully below the ~20 ms most shared-mode paths (including FlexASIO's default) settle for.
- **2. WDM-KS fallback** — used automatically only if a device doesn't support IAudioClient3's low-latency path, or if the user explicitly opts into it for maximum headroom and accepts single-app exclusivity (this preserves an ASIO4ALL-style option for users who want it, rather than removing the choice).
- **3. Standard WASAPI shared mode** — final fallback if neither of the above is available, so the driver still works on any device rather than failing outright.
- **User-visible mode indicator:** the control panel shows which backend is actually active for the current device — unlike the surveyed competitors, the user isn't left guessing why performance differs between two different audio interfaces.

2.3 Configurable Buffer Size Design

Design: ASIOGetBufferSize reports a live-queried range — not a hardcoded value — sourced from whichever backend is active for that device (IAudioClient3's reported engine period range, or the WDM-KS device's own reported range). Granularity is set to the finest step the backend actually supports. This directly targets the Steinberg complaint: where Steinberg's driver returns one fixed number, this driver always returns a genuine, honestly-sourced range, re-queried per sample rate.

- Selection is available both through the DAW's own ASIO buffer-size UI (when it has one) and through the standalone control panel (for hosts that don't).

- Changing the buffer size triggers a clean stop/reconfigure/restart of the stream — fast, but not promised as invisible to the host, which would overstate what the ASIO buffer-switch model supports.
- Selecting a size the live measurement subsystem flags as underrun-prone shows a warning with the measured headroom, rather than silently blocking or silently allowing a bad choice.

2.4 Multi-Client Shared Audio — Design

Because the default backend (2.2) is shared-mode, other applications — a browser, Discord, system notification sounds — continue to play audio normally while the DAW has the driver open. This is the direct, structural fix for the ASIO4ALL complaint, achieved as a side effect of the backend choice rather than as a bolted-on feature.

The tradeoff is stated honestly rather than hidden: shared mode, even with IAudioClient3's tighter negotiated period, has a latency floor that depends on the device's WaveRT miniport quality (1.4). On hardware with a good driver this can approach single-digit milliseconds; on weaker integrated hardware it may only modestly beat the standard 20 ms floor. The WDM-KS opt-in (2.2, step 2) remains available for users who want ASIO4ALL/FlexASIO-Exclusive-class latency and are willing to trade away multi-client audio for it — the point is that this driver makes that an explicit, informed choice instead of the only option.

2.5 Scheduling — Design Recap

As established in the prior full research pass: real-time threads register with MMCSS under the “Pro Audio” category and are pinned to P-core CPU Sets where available. This matters more here than in a simple fixed-buffer driver specifically because this driver is deliberately offering users small, user-selected buffer sizes — without this scheduling discipline, a low buffer size would glitch under load and reproduce exactly the kind of unreliability that gives universal ASIO drivers their reputation.

PART 3 — IMPLEMENTATION

Each phase has an explicit gate. The project does not advance until the previous phase's success criteria are met on real hardware — not assumed.

Phase	Objective	Success criteria	Decision
Phase 1 — Baseline	Measure actual round-trip latency and multi-client behavior of FL Studio ASIO, FlexASIO (default config), ASIO4ALL, and Steinberg's built-in driver on every test device, using a loopback measurement tool (not buffer-size arithmetic).	Reproducible measured baseline for all four competitors on every test device.	No comparison claims are made anywhere else in this project until this baseline is real, measured data.
Phase 2 — ASIO shim + buffer protocol	Implement the IASIO COM interface and live-queried ASIOGetBufferSize/ASIOCreateBuffers per 2.3, per sample rate.	A test host can enumerate and select at least 4 distinct buffer sizes per sample rate and stream cleanly at each, on at least 2 different devices.	If granularity can't be made finer than fixed steps on a given backend, document it as hardware-dependent rather than forcing a false range.
Phase 3 — IAudioClient3-first negotiation	Implement the negotiation order from 2.2 (IAudioClient3 → WDM-KS → standard shared) with automatic per-device fallback.	Confirmed lower negotiated period than standard shared mode on at least 2 different devices, while another application plays audio simultaneously without interruption.	If a device shows no IAudioClient3 improvement, fall back cleanly and log it rather than forcing an unhelpful path.
Phase 4 — Control panel + mode toggle	Build the standalone UI: buffer-size selection, live range display, active-backend indicator, and the WDM-KS "max performance / single app" opt-in from 2.2.	A non-technical tester can see which mode is active, change buffer size, and understand the multi-client tradeoff without reading documentation.	Usability gate, not just functional.
Phase 5 — Scheduling hardening	Apply MMCSS Pro Audio + CPU-Sets P-core pinning specifically at the smallest 2–3 buffer sizes exposed in Phase 2.	Stable 30+ minute soak run at the smallest exposed buffer size with zero unrecovered dropouts on an idle system, for both the IAudioClient3 and WDM-KS paths.	If the smallest advertised size can't pass this soak test, raise the advertised minimum instead of shipping it.
Phase 6 — Head-to-head validation	Re-run the Phase 1 measurement tool against this driver, at matched buffer sizes, against all four named competitors, on every test device — including a check that other audio keeps playing.	Measured round-trip latency at the smallest stable buffer size is lower than at least FL Studio ASIO, FlexASIO's default config, and Steinberg's built-in driver on the same hardware, while other audio keeps playing (unlike ASIO4ALL).	Report per-device, honestly — including any device where the gap is small, per the hardware-dependency caveat in 1.4.

PART 4 — FINAL PROJECT PLAN

How Difficult Is This, Honestly

This is a pure user-mode COM component — no kernel driver, no WHQL signing, no driver-store install. That removes the hardest and slowest part of Windows audio driver development entirely. The genuinely novel piece of engineering — implementing IAudioClient3's low-latency shared-mode calls — is not exotic research; it's a well-documented, years-old Microsoft API that a mature, widely-used open-source project has simply never gotten around to implementing. That combination (bounded scope + a real, specific, citable gap) is what makes this a credible solo/small-team project rather than an open-ended research problem.

Milestone	Realistic effort	Why
Working MVP (one backend, fixed device, one DAW)	A few weeks of focused work for someone fluent in C++/COM and Windows Core Audio	The ASIO callback contract and WASAPI/WDM-KS APIs are both well documented; FlexASIO's open-source backend code is a direct reference.
IAudioClient3 negotiation + fallback chain working reliably across devices	Additional weeks — the core new engineering	This is the part nobody else has shipped; expect iteration once you have several different physical devices to test against.
Robust across multiple DAWs and a range of consumer hardware	Months of incremental iteration, not a one-time build	This is exactly where FlexASIO/KoordASIO have spent years fixing edge cases — DAW quirks and hardware/driver variability are the long pole, not the core algorithm.

Major Risks

- **Hardware/driver dependency (1.4):** IAudioClient3's real achievable minimum period varies by device; the improvement over standard shared mode will not be uniform, and this must be reported per-device rather than as a blanket number.
- **Overclaiming risk:** the honest, defensible claim is beating FL Studio ASIO, FlexASIO's default configuration, ASIO4ALL's exclusivity, and Steinberg's fixed buffer — not beating co-designed vendor hardware ASIO drivers (RME, Focusrite, Universal Audio), which remain a higher bar this project isn't claiming to clear.
- **DAW compatibility risk:** each DAW negotiates with an ASIO driver slightly differently; budget real testing time across Cubase, Ableton, FL Studio, and Reaper rather than assuming one implementation works everywhere.
- **Licensing choice:** since October 2025 the ASIO SDK is available under GPLv3 or the historical proprietary license. GPLv3 removes the need to sign Steinberg's proprietary agreement but requires derivative works to also be GPLv3 — decide this deliberately based on how you intend to distribute the driver.
- **Scheduling risk:** advertising a buffer size lower than Phase 5's soak test actually supports reproduces the exact “unreliable universal driver” reputation this project is trying to improve on.

Final Answer

Build the buffer-size protocol (Phase 2) and the IAudioClient3-first negotiation (Phase 3) first — together they are the direct, mechanical fix for all four named complaints, because they're the same underlying gap wearing four different names. Reuse the ASIO callback contract, FlexASIO's open-source WASAPI/WDM-KS backend code as a direct reference, and the MMCSS/CPU-Sets scheduling discipline from the prior research — none of this requires new

invention. The one piece of genuinely new engineering — IAudioClient3's shared-mode negotiation — is exactly the piece that is verified, specific, and currently unclaimed by every driver you named. Avoid overclaiming beyond that: this project can credibly beat FL Studio ASIO, FlexASIO's out-of-the-box config, ASIO4ALL's exclusivity, and Steinberg's fixed buffer — not co-designed vendor hardware, which stays out of scope.